■ FastAPI

Interview-Ready Revision Sheet

Based on CampusX YouTube Playlist • FastAPI for Machine Learning

Topics: APIs · HTTP Methods · CRUD · Pydantic · Path & Query Params · ML Serving · Docker · AWS Deploy

01 | What is an API?

Definition:

An **API (Application Programming Interface)** is a mechanism that allows the **frontend** (user interface) to communicate with the **backend** (logic + database) **without directly accessing backend code or the database**. It acts as a secure middle layer, exposing controlled access points called **endpoints** (e.g., /home, /about, /predict).

Architecture of a Web Application:

Frontend	What the user sees and interacts with (HTML, React, etc.)
API Layer	Secure interface that handles requests/responses (FastAPI, Flask, etc.)
Backend	Application logic — processing, ML inference, business rules
Database	Stores persistent data (PostgreSQL, MongoDB, SQLite, etc.)

■ WHY APIs?	APIs protect your database by never exposing it directly. Only the data the API chooses to return is visible to the client.

Static vs Dynamic Websites:

Static Website	Content does not change. No user interaction, no DB. E.g., calendar page showing fixed dates.
Dynamic Website	Content changes based on user input. Supports CRUD. Uses APIs to interact with backend/DB.

02 | HTTP Methods & Status Codes

HTTP Methods → CRUD Mapping:

HTTP Method	CRUD Operation	Description	Status Code
GET	Read	Retrieve data from server	200 OK
POST	Create	Send new data to server	201 Created
PUT	Update	Replace entire resource	200 OK

PATCH	Update	Partial update of resource	200 OK
DELETE	Delete	Remove a resource	200/204

HTTP Status Codes (Interview Favourites):

Code	Meaning	Common Scenario
200	OK	Successful GET / PUT / DELETE
201	Created	Successful POST (resource created)
204	No Content	DELETE success with no body
400	Bad Request	Invalid input / validation error
401	Unauthorized	Missing / invalid auth token
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource does not exist
422	Unprocessable Entity	Pydantic validation failure (FastAPI default)
500	Internal Server Error	Unhandled server-side exception

■■ INTERVIEW TIP — FastAPI returns **422 Unprocessable Entity** by default when Pydantic validation fails — not 400. Remember this for interviews!

03 | Why FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of **Starlette** (ASGI framework for async) and **Pydantic** (data validation via Python type hints).

Speed	On par with NodeJS and Go. Built on Starlette (ASGI) + Pydantic + Rust-based tooling.
Async Support	Supports async/await natively — handles multiple requests simultaneously without blocking.
Auto Docs	Generates Swagger UI (/docs) and ReDoc (/redoc) automatically from your code.
Type Safety	Uses Python type hints → fewer bugs, better IDE support, automatic validation.
Pydantic	Handles input validation, serialization, and error messages automatically.
Standards-based	Fully compatible with OpenAPI (Swagger) and JSON Schema.

Setup Commands:

```
# Create virtual environment
python -m venv myenv

# Activate (Windows)
myenv\Scripts\activate

# Activate (Linux/Mac)
source myenv/bin/activate

# Install FastAPI + Uvicorn
pip install fastapi uvicorn pydantic

# Run the app
uvicorn main:app --reload
# main = filename (main.py), app = FastAPI instance, --reload = auto restart on save
```

■ ASGI vs WSGI	**ASGI vs WSGI:** WSGI (Flask, Django) is synchronous — one request at a time. ASGI (FastAPI, Starlette) is asynchronous — handles concurrent requests efficiently.

04 | First FastAPI App

```
from fastapi import FastAPI

app = FastAPI()       # Create the FastAPI instance

@app.get("/")         # Decorator: defines a GET endpoint at root URL "/"
def hello():
    return {"message": "Hello, World!"}   # Auto-converted to JSON

@app.get("/about")
def about():
    return {"message": "This is the about section."}

# After running: visit http://127.0.0.1:8000/docs  →  Swagger UI auto-generated!
```

@app.get("/")	Creates a GET endpoint at the root URL /
return dict	FastAPI auto-serializes Python dicts to JSON responses
/docs	Auto-generated interactive Swagger UI documentation
/redoc	Alternative ReDoc documentation UI

05 | Pydantic & Data Validation

Definition:

Pydantic is a data validation and settings management library for Python. FastAPI uses Pydantic models to define the **schema** (shape) of request bodies and response data. Pydantic enforces types, validates constraints, and auto-generates API documentation schemas.

Basic Pydantic Model:

```
from pydantic import BaseModel, Field, computed_field
from typing import Annotated, Literal, Optional

class Patient(BaseModel):
    id:     Annotated[str,   Field(..., description="Patient ID", examples=["P001"])]
    name:   Annotated[str,   Field(..., description="Full name")]
    city:   Annotated[str,   Field(..., description="City")]
    age:    Annotated[int,   Field(..., gt=0, lt=120)]      # gt = greater than, lt = less than
    gender: Annotated[Literal["male","female","others"], Field(...)]
    height: Annotated[float, Field(..., gt=0)]              # in metres
    weight: Annotated[float, Field(..., gt=0)]              # in kgs

    @computed_field                 # Dynamically computed — not in request body
    @property
    def bmi(self) -> float:
        return round(self.weight / (self.height ** 2), 2)

    @computed_field
    @property
    def verdict(self) -> str:
        if self.bmi < 18.5: return "Underweight"
        elif self.bmi < 25: return "Normal"
        elif self.bmi < 30: return "Overweight"
        else: return "Obese"
```

Key Pydantic Concepts:

BaseModel	All Pydantic models inherit from this. Defines schema, validates input automatically.
Field()	Adds metadata, validation constraints (gt, lt, ge, le), and descriptions to fields.
Annotated[]	Wraps a type with additional metadata/constraints. Preferred modern approach in FastAPI.
Literal[]	Restricts a field to specific allowed string values. E.g., Literal["male","female"].
Optional[]	Makes a field optional (can be None). Used in update models for partial updates.
@computed_field	Dynamically computed property. Not accepted from user input — calculated server-side.
model_dump()	Converts Pydantic model to Python dict. Use exclude=["id"] to omit fields.
model_dump(exclude_unset=True)	Returns only fields explicitly set by user — crucial for PATCH/PUT partial updates.

Update Model (for partial PUT):

```
class PatientUpdate(BaseModel):
    # All fields Optional — user can update any subset
    name:   Annotated[Optional[str],   Field(default=None)]
    city:   Annotated[Optional[str],   Field(default=None)]
    age:    Annotated[Optional[int],   Field(default=None, gt=0)]
    gender: Annotated[Optional[Literal["male","female"]], Field(default=None)]
    height: Annotated[Optional[float], Field(default=None, gt=0)]
    weight: Annotated[Optional[float], Field(default=None, gt=0)]
```

■ INTERVIEW INSIGHT **Why a separate update model?** The original Patient model has required fields — sending a PUT would force all fields. PatientUpdate makes all optional so user can update just age=25 without resending everything.

06 | Path & Query Parameters

Path Parameter	Part of the URL path itself. Used to identify a specific resource. E.g., /patient/{patient_id}
Query Parameter	Appended after ? in URL. Used to filter/sort/control results. E.g., /sort?by=bmi■=desc

```
from fastapi import FastAPI, Path, Query, HTTPException

app = FastAPI()

# ■■ PATH PARAMETER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.get("/patient/{patient_id}")
def view_patient(
    patient_id: str = Path(...,
        description="ID of the patient in the DB",
        example="P001"           # shows up in /docs
    )
):
    data = load_data()
    if patient_id in data:
        return data[patient_id]
    raise HTTPException(status_code=404, detail="Patient not found")

# ■■ QUERY PARAMETER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.get("/sort")
```

```
def sort_patients(
    sort_by: str = Query(..., description="Sort field: height, weight, bmi"),
    order:   str = Query("asc", description="asc or desc")
):
    valid_fields = ["height", "weight", "bmi"]
    if sort_by not in valid_fields:
        raise HTTPException(status_code=400, detail=f"Choose from {valid_fields}")
    if order not in ["asc", "desc"]:
        raise HTTPException(status_code=400, detail="Order must be asc or desc")

    data = load_data()
    reverse = (order == "desc")
    sorted_data = sorted(data.values(), key=lambda x: x.get(sort_by, 0), reverse=reverse)
    return sorted_data

# URL examples:
# GET /patient/P001          → Path param, fetches patient P001
# GET /sort?sort_by=bmi&order=desc  → Query params, sorts by BMI descending
```

■ TIP Path(…) and Query(…) add descriptions and validation to parameters — they also appear in Swagger UI docs automatically. The **...** means the parameter is required.

07 | Request Body & POST Endpoint

A **Request Body** carries data sent by the client to the server, typically in JSON format. FastAPI reads and validates the request body using a Pydantic model automatically.

```
from fastapi.responses import JSONResponse
import json

def load_data():
    with open("patients.json", "r") as f:
        return json.load(f)

def save_data(data):
    with open("patients.json", "w") as f:
        json.dump(data, f)

# ■■ POST → Create a new patient ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.post("/create")
def create_patient(patient: Patient):   # FastAPI parses & validates JSON body → Patient model
    data = load_data()

    if patient.id in data:
        raise HTTPException(status_code=400, detail="Patient already exists")

    # model_dump() → converts Patient to dict; exclude id since it is the key
    data[patient.id] = patient.model_dump(exclude=["id"])
    save_data(data)

    return JSONResponse(status_code=201, content={"message": "Patient created successfully"})

# Request body (JSON):
# {
#   "id": "P001",
#   "name": "Ravi Kumar",
#   "city": "Delhi",
#   "age": 28,
#   "gender": "male",
#   "height": 1.75,
#   "weight": 70
# }
```

08 | Full CRUD — PUT (Update) & DELETE

```
# ■■ PUT → Update patient ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.put("/edit/{patient_id}")
def update_patient(patient_id: str, patient_update: PatientUpdate):
    data = load_data()

    if patient_id not in data:
        raise HTTPException(status_code=404, detail="Patient not found")

    existing = data[patient_id]                          # existing patient dict

    # model_dump(exclude_unset=True) → only fields user actually provided
    updates = patient_update.model_dump(exclude_unset=True)

    for key, value in updates.items():
        existing[key] = value                            # apply each update

    # Re-validate through Pydantic (needed to recompute bmi + verdict)
    existing["id"] = patient_id
    updated_obj = Patient(**existing)
    data[patient_id] = updated_obj.model_dump(exclude=["id"])

    save_data(data)
    return JSONResponse(status_code=200, content={"message": "Patient updated"})

# ■■ DELETE → Remove patient ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.delete("/delete/{patient_id}")
def delete_patient(patient_id: str):
    data = load_data()

    if patient_id not in data:
        raise HTTPException(status_code=404, detail="Patient not found")

    del data[patient_id]
    save_data(data)
    return JSONResponse(status_code=200, content={"message": "Patient deleted"})
```

■ KEY INSIGHT

Why re-validate through Pydantic after PUT? If height or weight changed, the computed fields **bmi** and **verdict** must be recalculated. Re-creating Patient(**existing) triggers these @computed_field properties.

09 | HTTPException & Error Handling

HTTPException is FastAPI's way to return structured HTTP error responses. It immediately stops execution and sends a JSON error to the client.

```
from fastapi import HTTPException

# Raise 404 when resource is not found
raise HTTPException(status_code=404, detail="Patient not found")

# Raise 400 for invalid user input
raise HTTPException(status_code=400, detail="Patient already exists")

# Response JSON format:
# { "detail": "Patient not found" }

# ■■ Custom Exception Handler ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from fastapi import Request
from fastapi.responses import JSONResponse
```

```
@app.exception_handler(404)
async def not_found_handler(request: Request, exc: HTTPException):
    return JSONResponse(
        status_code=404,
        content={"error": "Resource not found", "path": str(request.url)}
    )
```

status_code	HTTP status code to return (400, 404, 422, 500, etc.)
detail	Human-readable error message returned in JSON body as {"detail": "..."}
headers	Optional dict of extra response headers to include

10 | Response Models & JSONResponse

```
from fastapi.responses import JSONResponse
from pydantic import BaseModel

# ■■ Using response_model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class PatientOut(BaseModel):
    name:    str
    age:     int
    bmi:     float
    verdict: str

@app.get("/patient/{patient_id}", response_model=PatientOut)
def get_patient(patient_id: str):
    # FastAPI automatically filters response to only PatientOut fields
    ...

# ■■ Using JSONResponse directly ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.post("/create")
def create_patient(patient: Patient):
    ...
    return JSONResponse(
        status_code=201,
        content={"message": "Patient created successfully"}
    )

# response_model= strips fields NOT in output schema → security & clarity
# JSONResponse= gives full manual control over status code + body
```

11 | Async Programming in FastAPI

FastAPI supports **async/await** natively. Use **async def** when your endpoint involves I/O-bound operations (DB queries, HTTP calls, file reads). Synchronous **def** endpoints are still valid and run in a thread pool.

```
import asyncio
import httpx

# ■■ Sync endpoint (runs in thread pool automatically) ■■■■■■■■
@app.get("/sync")
def sync_endpoint():
    return {"type": "synchronous"}

# ■■ Async endpoint (non-blocking I/O) ■■■■■■■■■■■■■■■■■■■■■■■■
@app.get("/async")
async def async_endpoint():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/data")
    return response.json()
```

```
# ■■ Async with database ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@app.get("/patients")
async def get_all_patients():
    # With async DB (e.g. databases, asyncpg)
    result = await db.fetch_all("SELECT * FROM patients")
    return result

# Rule of thumb:
# → External API calls, DB queries → use async def
# → CPU-heavy ML inference, pure computation → use def (or run_in_executor)
```

def endpoint	Runs in thread pool. Safe for blocking I/O but less scalable.
async def endpoint	Runs in event loop. Non-blocking — best for I/O-heavy endpoints.
await	Suspends current coroutine until async operation completes. Must be inside async def.
asyncio	Python standard library for writing concurrent async code.

12 | Serving ML Models with FastAPI

The core use-case of the CampusX playlist: wrapping a trained ML model (pickle/joblib) inside a FastAPI endpoint for real-time predictions.

Step 1 — Train & Serialize Model:

```
# train.py
import pandas as pd
from sklearn.linear_model import LinearRegression
import pickle

df = pd.read_csv("insurance.csv")
# preprocessing ...
X = df[["age","bmi","children","smoker_encoded","region_encoded"]]
y = df["charges"]

model = LinearRegression()
model.fit(X, y)

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
print("Model saved!")
```

Step 2 — Define Pydantic Input/Output Schemas:

```
# schemas.py
from pydantic import BaseModel

class InsuranceInput(BaseModel):
    age:      int
    bmi:      float
    children: int
    smoker:   str     # "yes" or "no"
    region:   str     # "southwest", "southeast", etc.

class PredictionOutput(BaseModel):
    predicted_premium: float
    model_version:     str
```

Step 3 — FastAPI Prediction Endpoint:

```
# app.py
from fastapi import FastAPI
```

```
import pickle, numpy as np
from schemas import InsuranceInput, PredictionOutput

app = FastAPI()

# Load model once at startup (not inside the endpoint!)
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

SMOKER_MAP = {"yes": 1, "no": 0}
REGION_MAP = {"southwest": 0, "southeast": 1, "northwest": 2, "northeast": 3}

@app.post("/predict", response_model=PredictionOutput)
def predict(data: InsuranceInput):
    features = np.array([[
        data.age,
        data.bmi,
        data.children,
        SMOKER_MAP[data.smoker],
        REGION_MAP[data.region]
    ]])
    prediction = model.predict(features)[0]
    return PredictionOutput(
        predicted_premium=round(prediction, 2),
        model_version="v1.0"
    )

@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": model is not None}
```

■ PERFORMANCE TIP | Always load your model **at startup** (module-level or using @app.on_event("startup")), not inside the endpoint function. Loading inside the function would reload the model on every request — extremely slow!

Model Serialization Comparison:

pickle	Built-in Python. Suitable for sklearn models. Use with caution — not secure against untrusted files.
joblib	Preferred for large numpy arrays / sklearn pipelines. Faster for large models.
Keras	Use model.save("model.h5") or model.save("model.keras") + tf.keras.models.load_model().
ONNX	Framework-agnostic format. Export PyTorch/TF models for production serving.

13 | Dockerizing a FastAPI Application

Docker packages your FastAPI app + dependencies + model into a portable container that runs identically on any machine — dev laptop, CI server, or cloud.

Dockerfile:

```
# Dockerfile
FROM python:3.10-slim

WORKDIR /app

# Copy requirements first (Docker layer caching optimization)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy rest of application
COPY . .

# Expose port 8000
EXPOSE 8000

# Run FastAPI with Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt:

```
fastapi
uvicorn[standard]
pydantic
scikit-learn
numpy
pandas
pickle5
```

Docker Commands:

```
# Build the image
docker build -t fastapi-ml-app .

# Run container (map host port 8000 → container port 8000)
docker run -p 8000:8000 fastapi-ml-app

# Run in background (detached)
docker run -d -p 8000:8000 fastapi-ml-app

# List running containers
docker ps

# Stop container
docker stop <container_id>

# Docker Compose (multi-service)
docker-compose up --build
```

FROM	Base image. python:3.10-slim is lightweight.
WORKDIR	Sets working directory inside container.
COPY	Copies files from host into container.
RUN	Executes commands during image build (install dependencies).
EXPOSE	Documents which port the container listens on (informational).
CMD	Default command to run when container starts. Use 0.0.0.0 to accept external connections.

14 | Deploying FastAPI on AWS

Deployment Flow (AWS EC2 + Docker):

```
# ■■ Step 1: Push Docker image to Docker Hub ■■■■■■■■■■■■■■■■■■■■
docker login
docker tag fastapi-ml-app yourdockerhubuser/fastapi-ml-app:v1
docker push yourdockerhubuser/fastapi-ml-app:v1

# ■■ Step 2: Launch EC2 instance on AWS ■■■■■■■■■■■■■■■■■■■■■■■■
# - Go to AWS Console → EC2 → Launch Instance
# - Choose Ubuntu 22.04 AMI
```

```
# - Open port 8000 in Security Group (Inbound rules)

# ■■ Step 3: SSH into EC2 and install Docker ■■■■■■■■■■■■■■■■■■■■
ssh -i your-key.pem ubuntu@<ec2-public-ip>
sudo apt-get update
sudo apt-get install -y docker.io
sudo systemctl start docker

# ■■ Step 4: Pull and run your container ■■■■■■■■■■■■■■■■■■■■■■■■
sudo docker pull yourdockerhubuser/fastapi-ml-app:v1
sudo docker run -d -p 8000:8000 yourdockerhubuser/fastapi-ml-app:v1

# ■■ Access your API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# http://<ec2-public-ip>:8000/docs   ← Swagger UI
# http://<ec2-public-ip>:8000/predict ← Prediction endpoint
```

EC2	Elastic Compute Cloud — virtual server on AWS where Docker container runs.
Security Group	AWS firewall rules. Must allow inbound TCP on port 8000 for public access.
Docker Hub	Public container registry. Push image here, pull from EC2.
ECR	AWS Elastic Container Registry — private alternative to Docker Hub.
ECS/Fargate	AWS managed container orchestration — no EC2 management needed.

15 | Complete CampusX Patient API — Full Code

This is the complete working code from the CampusX FastAPI playlist — covers all CRUD operations, Pydantic models, path/query params, error handling, and computed fields.

```
from fastapi import FastAPI, Path, HTTPException, Query
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field, computed_field
from typing import Annotated, Literal, Optional
import json

app = FastAPI()

class Patient(BaseModel):
    id:     Annotated[str,   Field(..., description="Patient ID", examples=["P001"])]
    name:   Annotated[str,   Field(..., description="Name")]
    city:   Annotated[str,   Field(..., description="City")]
    age:    Annotated[int,   Field(..., gt=0, lt=120)]
    gender: Annotated[Literal["male","female","others"], Field(...)]
    height: Annotated[float, Field(..., gt=0)]
    weight: Annotated[float, Field(..., gt=0)]

    @computed_field
    @property
    def bmi(self) -> float:
        return round(self.weight / (self.height ** 2), 2)

    @computed_field
    @property
    def verdict(self) -> str:
        if self.bmi < 18.5: return "Underweight"
        elif self.bmi < 25: return "Normal"
        elif self.bmi < 30: return "Overweight"
        else: return "Obese"

class PatientUpdate(BaseModel):
    name:   Annotated[Optional[str],   Field(default=None)]
    city:   Annotated[Optional[str],   Field(default=None)]
    age:    Annotated[Optional[int],   Field(default=None, gt=0)]
    gender: Annotated[Optional[Literal["male","female"]], Field(default=None)]
    height: Annotated[Optional[float], Field(default=None, gt=0)]
    weight: Annotated[Optional[float], Field(default=None, gt=0)]

def load_data():
    with open("patients.json", "r") as f:
        return json.load(f)

def save_data(data):
    with open("patients.json", "w") as f:
        json.dump(data, f)

@app.get("/")
def hello():
    return {"message": "Patient Management System API"}

@app.get("/view")
def view():
    return load_data()

@app.get("/patient/{patient_id}")
def view_patient(patient_id: str = Path(..., description="Patient ID", example="P001")):
    data = load_data()
    if patient_id in data:
        return data[patient_id]
    raise HTTPException(status_code=404, detail="Patient not found")
```

```
@app.get("/sort")
def sort_patients(
    sort_by: str = Query(..., description="height, weight, or bmi"),
    order: str = Query("asc", description="asc or desc")
):
    valid = ["height", "weight", "bmi"]
    if sort_by not in valid:
        raise HTTPException(status_code=400, detail=f"Choose from {valid}")
    if order not in ["asc", "desc"]:
        raise HTTPException(status_code=400, detail="Order must be asc or desc")
    data = load_data()
    return sorted(data.values(), key=lambda x: x.get(sort_by, 0), reverse=(order=="desc"))

@app.post("/create")
def create_patient(patient: Patient):
    data = load_data()
    if patient.id in data:
        raise HTTPException(status_code=400, detail="Patient already exists")
    data[patient.id] = patient.model_dump(exclude=["id"])
    save_data(data)
    return JSONResponse(status_code=201, content={"message": "Patient created"})

@app.put("/edit/{patient_id}")
def update_patient(patient_id: str, patient_update: PatientUpdate):
    data = load_data()
    if patient_id not in data:
        raise HTTPException(status_code=404, detail="Patient not found")
    existing = data[patient_id]
    for k, v in patient_update.model_dump(exclude_unset=True).items():
        existing[k] = v
    existing["id"] = patient_id
    data[patient_id] = Patient(**existing).model_dump(exclude=["id"])
    save_data(data)
    return JSONResponse(status_code=200, content={"message": "Patient updated"})

@app.delete("/delete/{patient_id}")
def delete_patient(patient_id: str):
    data = load_data()
    if patient_id not in data:
        raise HTTPException(status_code=404, detail="Patient not found")
    del data[patient_id]
    save_data(data)
    return JSONResponse(status_code=200, content={"message": "Patient deleted"})
```

16 | Top Interview Questions & Answers

■ Q1. What is FastAPI and what makes it fast?

→ FastAPI is a modern Python web framework for building APIs. It's fast because: (1) built on Starlette — an ASGI async framework; (2) uses Pydantic for ultra-fast data validation; (3) supports async/await for concurrent I/O handling; (4) leverages Rust-based tooling. Performance is comparable to NodeJS and Go.

■ Q2. What is the difference between ASGI and WSGI?

→ WSGI (Web Server Gateway Interface) is synchronous — handles one request at a time (Flask, Django). ASGI (Asynchronous Server Gateway Interface) supports async/await and handles multiple concurrent requests without blocking (FastAPI, Starlette). ASGI is essential for real-time apps, WebSockets, and ML inference APIs under load.

■ Q3. What is Pydantic and why does FastAPI use it?

→ Pydantic is a Python data validation library. FastAPI uses it to: (1) auto-validate incoming request data against defined schemas; (2) serialize response data; (3) auto-generate OpenAPI/JSON Schema docs. Any validation error returns a 422 Unprocessable Entity response with clear error details.

■ Q4. What is the difference between Path parameter and Query parameter?

→ Path parameter is embedded in the URL path (/patient/{id}) — used to identify a specific resource. Query parameter follows '?' in the URL (/sort?sort_by=bmi■=desc) — used to filter, sort, or modify results. Path params are required; query params can have defaults.

■ Q5. What does model_dump(exclude_unset=True) do?

→ It converts a Pydantic model to a dict including only the fields that were explicitly set by the user (not defaults). Crucial for partial updates (PUT/PATCH) — allows updating just age=25 without requiring all other fields to be re-sent.

■ Q6. What is a computed_field in Pydantic?

→ A @computed_field is a read-only property that is dynamically calculated from other model fields. It is included in model output (model_dump()) but NOT accepted from user input. Example: bmi is computed from height and weight.

■ Q7. How do you load an ML model in FastAPI efficiently?

→ Load the model at module level (outside any function) or in an @app.on_event('startup') handler. Never load inside the endpoint function — this reloads the model on every request, causing massive latency.

■ Q8. What is the purpose of Dockerfile CMD vs RUN?

→ RUN executes commands at image build time (install packages, setup). CMD defines the default command at container runtime (start the server). There can be multiple RUN but only one effective CMD. Example: CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"].

■ Q9. Why use --host 0.0.0.0 in Docker?

→ By default, Uvicorn binds to 127.0.0.1 (localhost) — only accessible inside the container. Using 0.0.0.0 binds to all network interfaces, allowing external connections (e.g., from your host machine or the internet via port mapping).

■ Q10. What HTTP status code does FastAPI return for validation errors?

→ 422 Unprocessable Entity — not 400. FastAPI returns this automatically when a Pydantic validation fails (e.g., passing a string where an int is expected). The response body contains detailed field-level error messages.

■ Q11. What is the difference between PUT and PATCH?

→ PUT replaces the entire resource — all fields must be provided. PATCH partially updates — only changed fields are sent. In practice, FastAPI doesn't enforce this distinction; it's a convention. The PatientUpdate model with all Optional fields enables partial updates via PUT.

■ Q12. How does FastAPI generate automatic documentation?

→ FastAPI reads your Python type hints, Pydantic models, and docstrings to auto-generate OpenAPI 3.0 spec. This is served as Swagger UI at /docs (interactive) and ReDoc at /redoc. No extra code needed.

17 | Quick Reference Cheat Sheet

Decorator	Purpose	Example
`@app.get("/path")`	GET endpoint — read data	`@app.get("/patients")`
`@app.post("/path")`	POST endpoint — create resource	`@app.post("/create")`
`@app.put("/path/{id}")`	PUT endpoint — update resource	`@app.put("/edit/{id}")`

`@app.delete("/path/{id}")`	DELETE endpoint — remove resource	`@app.delete("/delete/{id}")`
`Path(...)`	Path parameter with validation	`patient_id: str = Path(...)`
`Query(...)`	Query parameter with validation	`sort_by: str = Query(...)`
`HTTPException`	Raise HTTP error response	`raise HTTPException(404, ...)`
`BaseModel`	Pydantic request/response schema	`class Item(BaseModel): ...`
`Field()`	Field metadata + constraints	`Field(..., gt=0, lt=120)`
`@computed_field`	Computed read-only field	`@property def bmi(self)...`
`model_dump()`	Pydantic model → dict	`patient.model_dump()`
`model_dump(exclude_unset=True)`	Only user-provided fields	`update.model_dump(exclude_unset=True)`
`JSONResponse()`	Custom status + body response	`JSONResponse(201, content={...})`
`response_model=`	Filter and validate response output	`@app.get("/", response_model=Out)`